

Ok.
Código verificado.

Ok. Boa
a tarde
corrente.

PROJETO 1: FAMILIARIZAÇÃO COM MÁQUINA VIRTUAL E SINCRONIZAÇÃO DE PROCESSOS

Gustavo Borges Peres da Silva¹, Michael P. C. Silva¹, Rafael S. Melo¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

{gustavo.silva, michael.silva}@discente.ufcat.edu.br,
rafinhalelograma@hotmail.com

1. Tarefa 1: preparação do sistema

1.1. Descrição

Neste projeto, optamos por adotar um ambiente de trabalho personalizado, considerando que nossos computadores possuem configurações de hardware de alto desempenho. Com processadores avançados, placas gráficas de última geração, uma quantidade significativa de memória RAM e armazenamento SSD NVMe, nossos sistemas não apenas atendem, mas superam as demandas de desempenho exigidas por este projeto. Além disso, todos os hosts serão configurados utilizando o Windows 11 Pro, com virtualização ativada na BIOS e tecnologias que permitem a execução eficiente de máquinas virtuais, melhorando o desempenho e a eficiência do sistema.

Em contrapartida, utilizamos o Oracle VirtualBox 7.0 para criar a máquina virtualizada, denominada convidado (guest), que opera com o sistema operacional Linux Debian 12.8.0 (bookworm) que seria a versão mais recente durante a escrita deste relatório. A instalação foi realizada utilizando a imagem do instalador via rede, uma escolha estratégica que permite o download dos pacotes do sistema durante o processo de instalação. Essa opção foi especialmente vantajosa devido à alta velocidade de conexão com a internet disponível no momento da instalação..

1.2. Passo-a-passo da configuração da Máquina Virtual

1.2.1. Instalação do Linux Debian

O Linux Debian 12 está disponível para download no seguinte endereço: <https://www.debian.org/distrib/netinst>. Optamos por baixar a versão netinst, que é a opção "CDs pequenos ou pendrives USB". Esta escolha permite uma instalação mais leve e flexível, baixando apenas os pacotes necessários durante o processo de instalação.

Para a virtualização do Debian, escolhemos utilizar o Oracle VirtualBox como plataforma de hospedagem. Na figura 1 abaixo, compartilhamos a configuração inicial realizada por meio do assistente.

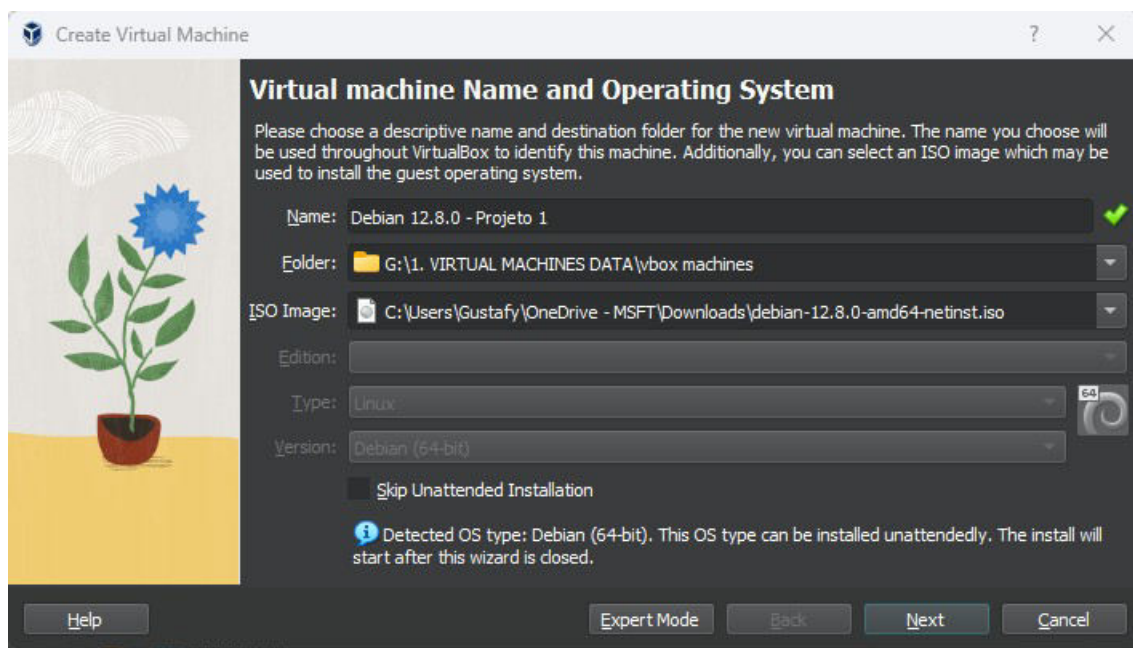


Figura 1. Criar máquina virtual

Na implementação do projeto, foi escolhido adotar uma estratégia eficiente por meio da instalação desassistida do sistema operacional convidado, utilizando a interface de configuração disponibilizada durante o assistente de instalação. Essa abordagem simplifica e automatiza o processo, eliminando a necessidade de intervenção manual repetitiva.

Durante a etapa de configuração desassistida no assistente de instalação, antecipamos a criação do usuário necessário para o sistema. Essa medida visa otimizar a eficiência do processo, garantindo padronização e consistência na implementação. Além disso, selecionamos a opção "Install in background" para evitar a necessidade de interação manual durante a instalação, simplificando o processo e reduzindo a complexidade deste relatório.

Ao definir previamente detalhes como nome de usuário e senha diretamente na interface de configuração, asseguramos que o sistema operacional convidado seja instalado com as configurações desejadas de forma rápida e precisa conforme a Figura 2 abaixo.

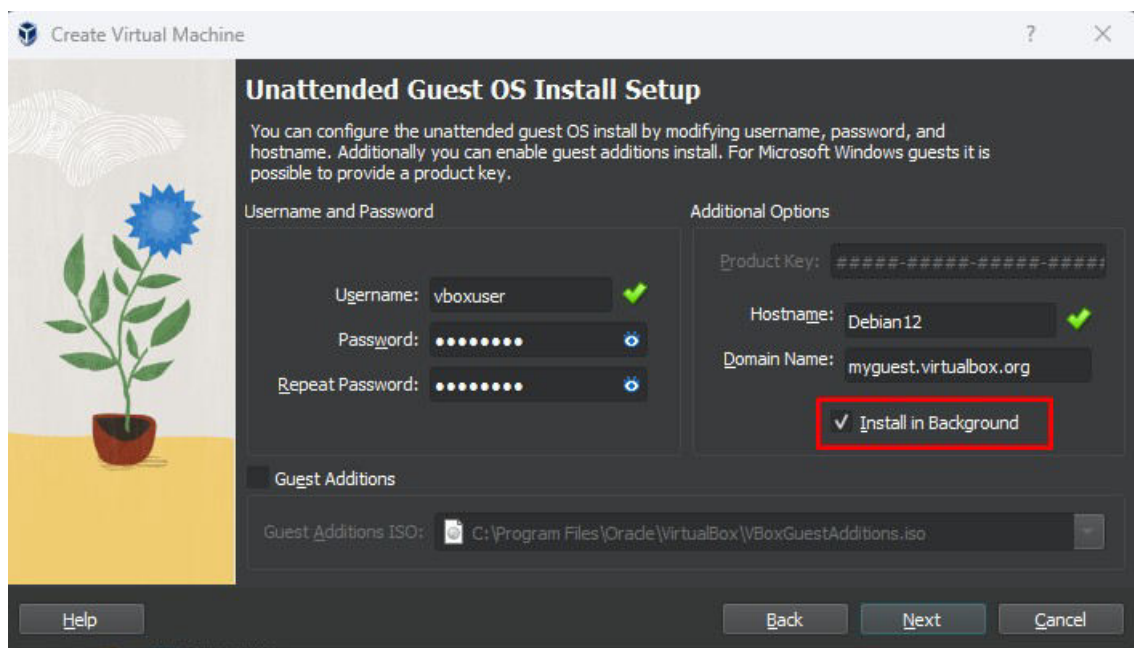


Figura 2. Configuração de Instalação Desassistida de SO guest.

Sobre a definição do hardware utilizado para a máquina guest, realizamos uma configuração adequada para evitar quedas de desempenho. Optamos por utilizar uma quantidade razoável de memória RAM, como 8GB (8*1024), e quase todos os núcleos do processador (4 núcleos) conforme a figura 3 abaixo.

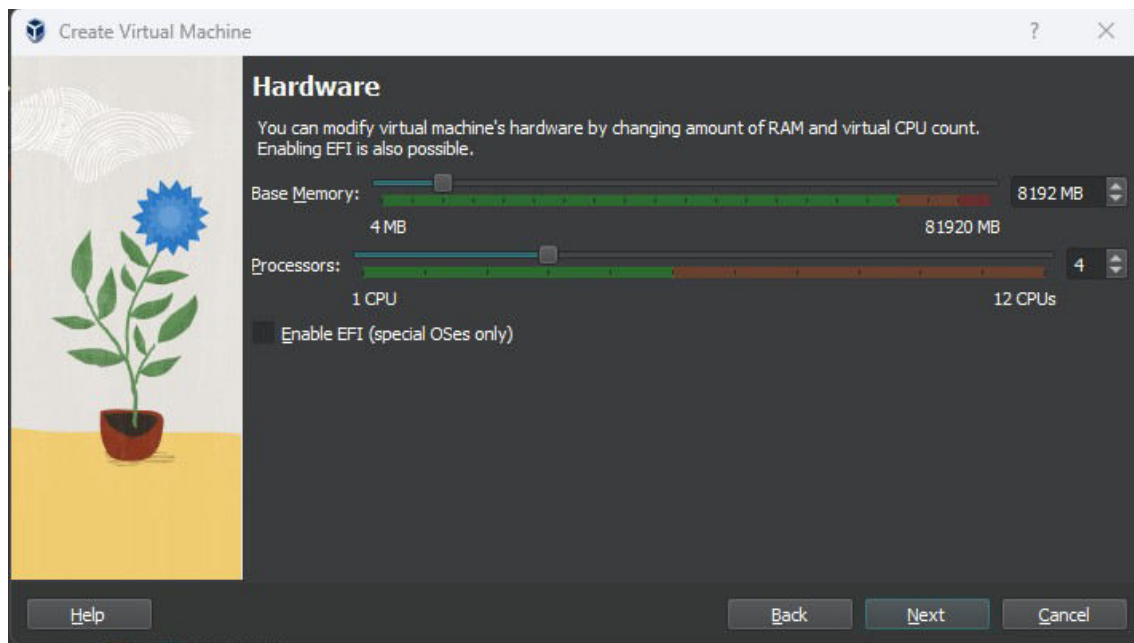


Figura 3. Hardware Guest.

Sobre o disco rígido da máquina virtual, consideramos um tamanho adequado para garantir que não teríamos problemas com armazenamento, aproveitando o espaço livre disponível na máquina host. Assim, utilizamos 15GB de armazenamento (15*1024)

pois iremos usar a interface gráfica gnome, conforme a figura 4 abaixo.

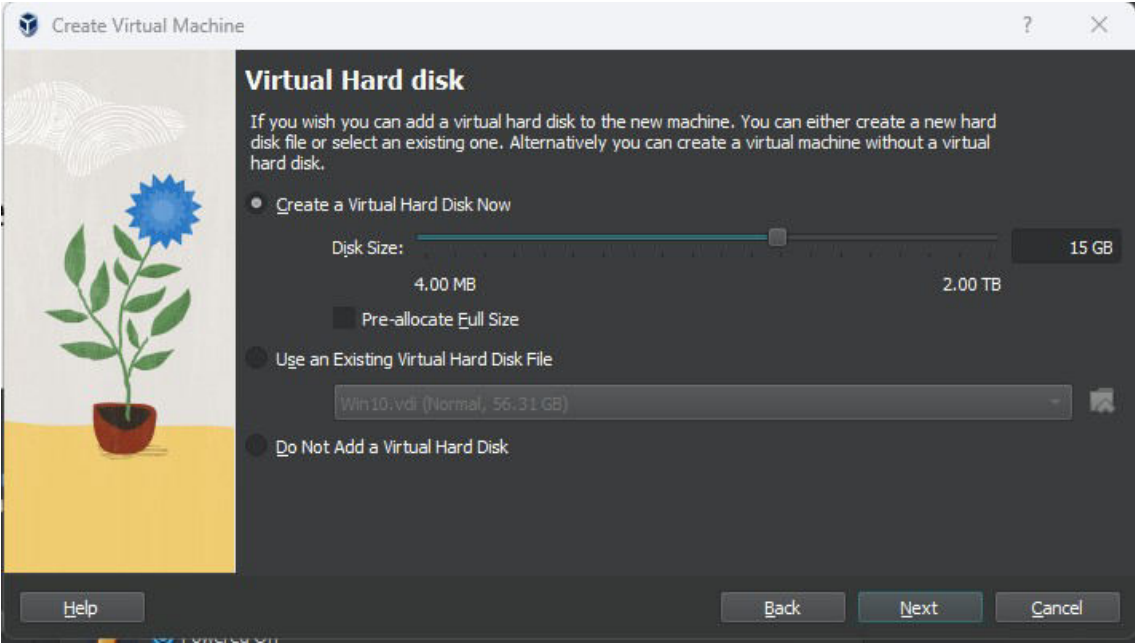


Figura 4. Armazenamento Guest.

Abaixo está o resumo da configuração da máquina.

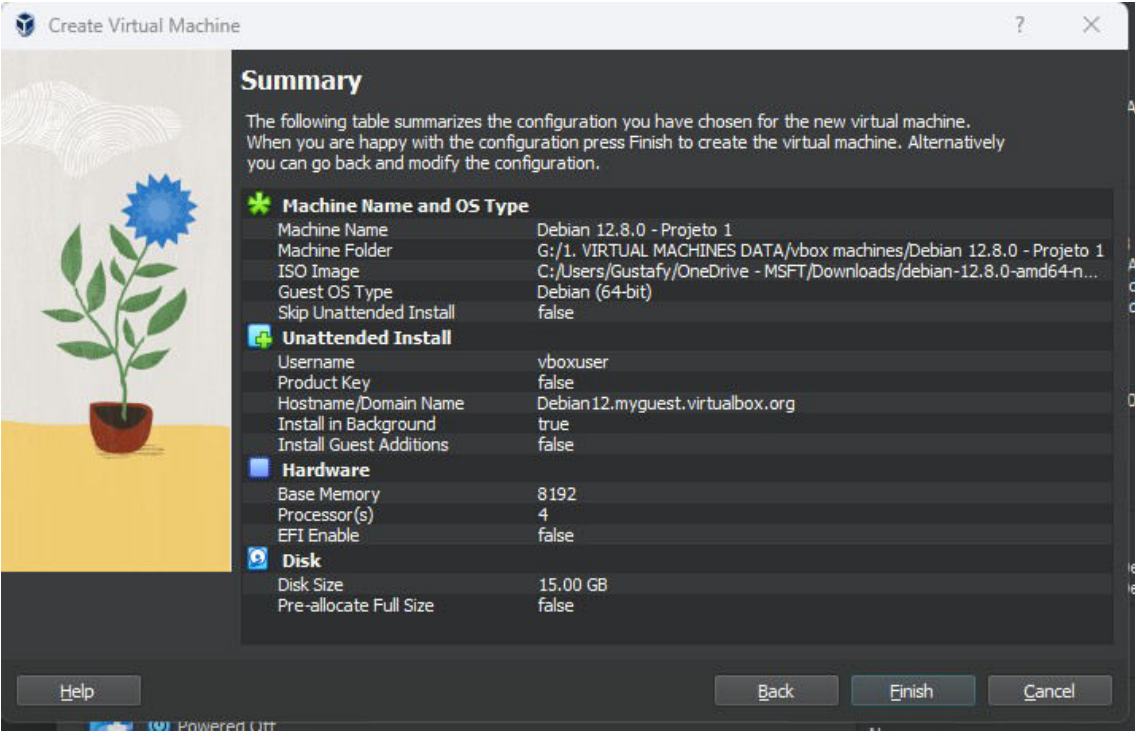


Figura 5. Resumo.

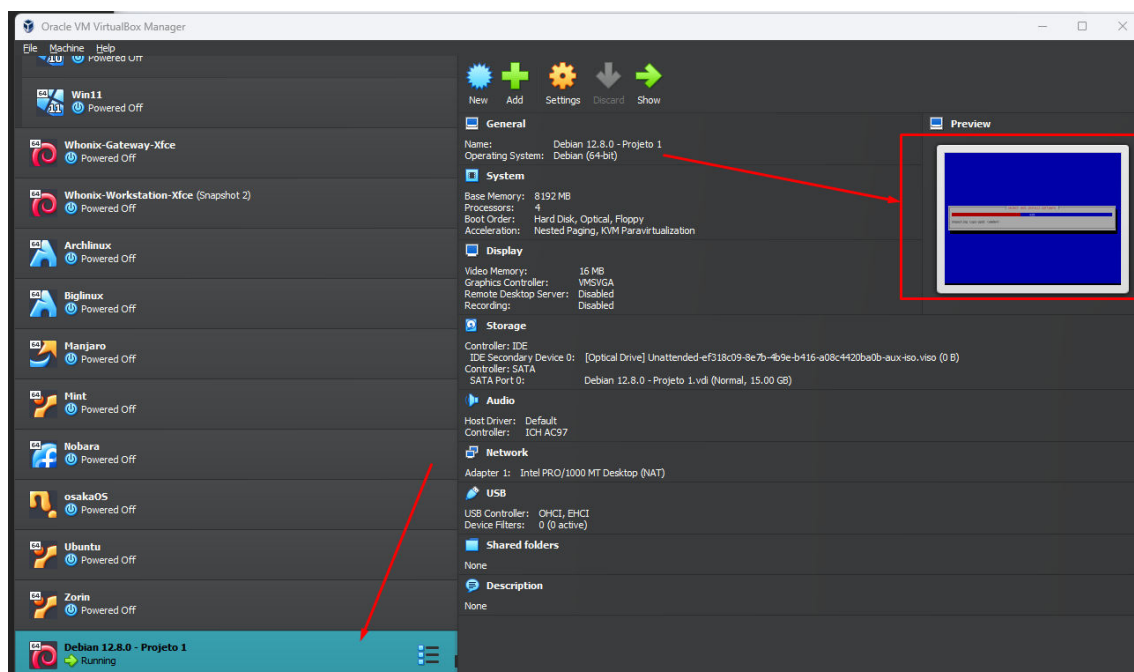


Figura 6. Processo de instalação desassistida e segundo plano.

Com aproximadamente 8 minutos a máquina virtual foi instalada com sucesso.

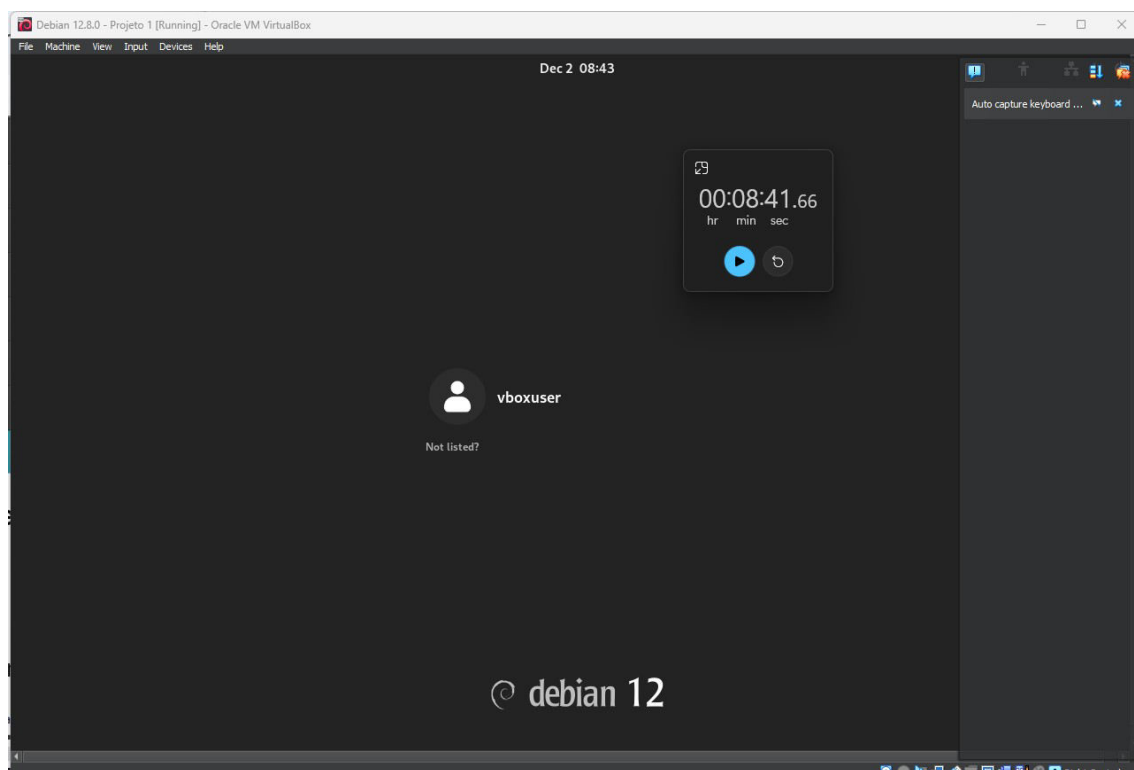


Figura 7. Máquina Virtual Instalada

1.2.2. Configuração da Máquina Virtual

Após a instalação do Debian, foi necessário atribuir o usuário vboxuser como administrador para que ele pudesse ter acesso ao root no arquivo /etc/sudoers (arquivo de permissões de usuários).

Passos:

1. **Entrar no usuário root:** Use o comando abaixo e digite a senha definida durante o setup do VirtualBox (neste caso, usamos a senha padrão changeme):

```
su
```

2. **Editar o arquivo de permissões (sudoers):** Abra o arquivo /etc/sudoers com o editor de texto nano:

```
sudo nano /etc/sudoers
```

3. **Adicionar permissões para o usuário:** Localize a linha que contém:

```
%sudo ALL=(ALL:ALL) ALL
```

Logo abaixo dessa linha, adicione a seguinte configuração (substitua nomeUsuario pelo nome do usuário, que no caso do projeto é vboxuser):

```
vboxuser ALL=(ALL) ALL
```

4. **Salvar e sair:** Pressione as seguintes combinações de teclas no editor nano:

```
Ctrl + O (Salvar alterações)
```

```
Enter (Confirmar)
```

```
Ctrl + X (Fechar o editor)
```

5. Sair do modo root: Após finalizar as alterações, digite:

```
exit
```

6. Alterar a senha do usuário vboxuser:

```
passwd vboxuser
```

Agora, o usuário vboxuser terá permissão de administrador para usar o sudo.

Adicionalmente, não foi necessário modificar o arquivo sources.list, um componente essencial na configuração do sistema Debian e suas distribuições relacionadas. Esse arquivo define os repositórios de onde o sistema obtém pacotes de software e atualizações.

Como optamos por instalar o Debian com conexão à internet, **o sistema configura automaticamente o sources.list durante a instalação**, garantindo acesso às versões mais recentes disponíveis. No caso do projeto, o arquivo foi configurado para a versão Debian Bookworm, apontando para o repositório de atualizações (bookworm-updates) com os componentes main e non-free-firmware. Essa configuração atende perfeitamente às necessidades atuais, proporcionando um ambiente estável, seguro e atualizado para as operações planejadas.

Ao abordar a **configuração da rede na máquina virtual**, é importante destacar que o VirtualBox oferece uma interface intuitiva para configurar a rede. O modo NAT (Network Address Translation) é uma opção comum e simples, mas, se precisarmos configurar o SSH no futuro, será necessário realizar o **redirecionamento de portas**, o que pode ser mais complexo. No entanto, essa abordagem garante maior segurança, pois evita expor diretamente o IP da máquina virtual à rede externa.

Por outro lado, o modo Bridge oferece maior simplicidade e rapidez, pois permite que a máquina virtual se comporte como um dispositivo independente na rede, com seu próprio IP. No entanto, essa configuração pode ser menos segura, já que a máquina virtual fica mais exposta à rede externa.

Diante disso, a escolha mais adequada foi habilitar a comunicação entre a máquina virtual e o sistema operacional hospedeiro através da configuração da placa de rede em modo NAT (Figura 8), com redirecionamento de portas (Figura 9). Essa configuração oferece uma boa combinação de segurança e funcionalidade para o ambiente de trabalho..

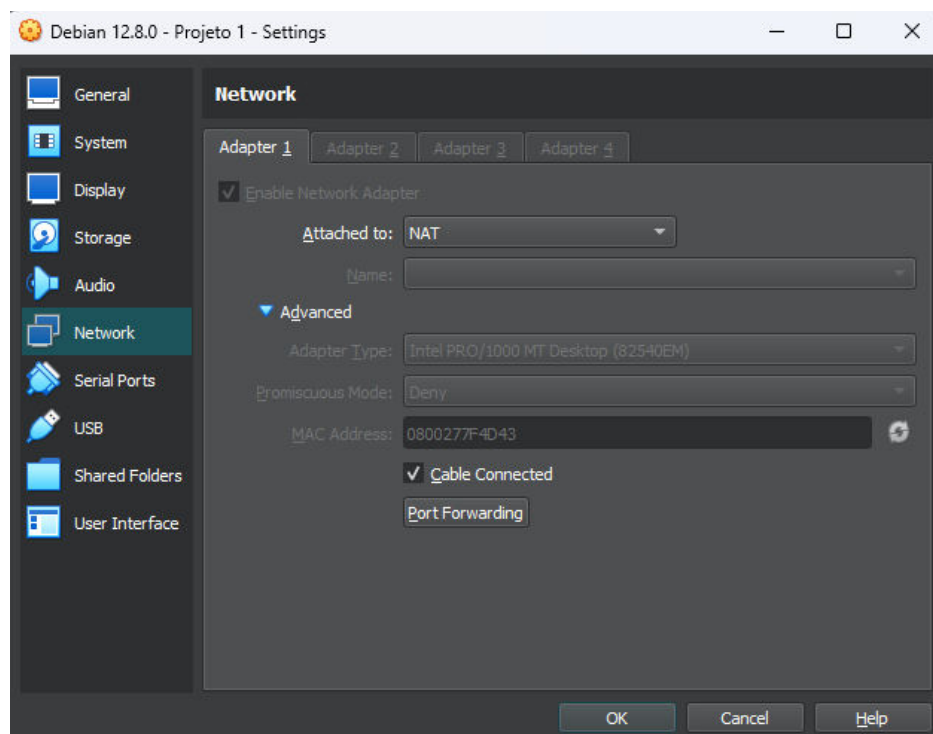


Figura 8. Configurando rede - NAT.

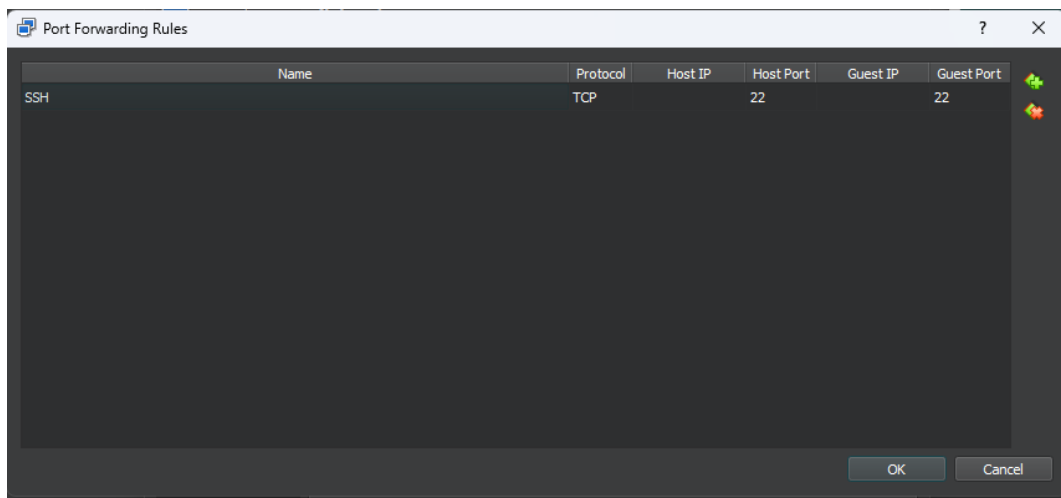


Figura 9. Configurando redirecionamento de portas.

Agora que a rede está configurada, vamos realizar alguns testes no Debian para verificar se tudo está funcionando corretamente. Os testes a seguir têm como objetivo confirmar a funcionalidade da rede. Vale ressaltar que antes de testarmos, precisamos instalar os seguintes pacotes:

```
sudo apt install speedtest-cli nload net-tools
```

Agora em relação aos testes, utilizando o comando “**ip a**”, podemos verificar as interfaces de rede e suas respectivas configurações. Durante o teste, obtemos o IP 10.0.2.15, conforme ilustrado na Figura 10.

```
vboxuser@Debian12: ~
vboxuser@Debian12:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:7f:4d:43 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86345sec preferred_lft 86345sec
    inet6 fe80::a00:27ff:fe7f:4d43/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
vboxuser@Debian12:~$
```

Figura 10. Interface de rede.

Foi utilizado também ferramentas específicas para medir a velocidade da internet

como por exemplo o ***speedtest-cli*** conforme na Figura 11 e o ***nload*** na Figura 12.

```
vboxuser@Debian12:~$ speedtest-cli
Retrieving speedtest.net configuration...
Testing from WGO Telecom (189.38.34.9)...
Retrieving speedtest.net server list...
Selecting best server based on ping...
Hosted by PHNet Telecom (Brasília) [264.46 km]: 43.894 ms
Testing download speed.....
.....
Download: 152.95 Mbit/s
Testing upload speed.....
.....
Upload: 198.19 Mbit/s
```

Figura 11. Teste com speedtest-cli.

```
vboxuser@Debian12: ~
Device enp0s3 [10.0.2.15] (1/2):
=====
Incoming:

Curr: 0.00 Bit/s
Avg: 0.00 Bit/s
Min: 0.00 Bit/s
Max: 0.00 Bit/s
Ttl: 958.74 MByte

Outgoing:

Curr: 0.00 Bit/s
Avg: 56.00 Bit/s
Min: 0.00 Bit/s
Max: 2.61 kBit/s
Ttl: 322.90 MByte
```

Figura 12. Teste com nload.

Adicionalmente, utilizamos o comando ***traceroute*** para verificar as rotas que os pacotes de dados tomam até um destino. No nosso exemplo, utilizamos o site do Google (www.google.com), conforme mostrado na Figura 13. O comando exibe os "hops" (saltos) intermediários, indicando os roteadores pelos quais os pacotes passam. Caso não haja resposta de alguns roteadores, isso pode ser devido a configurações de segurança, mas não significa necessariamente falha na conexão.

```
vboxuser@Debian12: ~  
vboxuser@Debian12:~$ traceroute www.google.com  
traceroute to www.google.com (172.217.29.68), 30 hops max, 60 byte packets  
1  _gateway (10.0.2.2)  0.209 ms  0.182 ms  0.422 ms  
2  * * *  
3  * * *  
4  * * *  
5  * * *  
6  * * *  
7  * * *  
8  * * *  
9  * * *  
10 * * *  
11 * * *  
12 * * *  
13 * * *  
14 * * *
```

Figura 13. Verificando rotas dos pacotes com traceroute.

E por último utilizamos o comando **netstat** para verificar as conexões de rede e o estado das portas conforme a Figura 14.

```
vboxuser@Debian12:~$ netstat -an  
Active Internet connections (servers and established)  
Proto Recv-Q Send-Q Local Address           Foreign Address         State  
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN  
tcp6       0      0 :::1:631                :::*                   LISTEN  
udp        0      0 0.0.0.0:56207           0.0.0.0:*  
udp        0      0 10.0.2.15:68            10.0.2.2:67           ESTABLISHED  
udp        0      0 0.0.0.0:5353            0.0.0.0:*  
udp6       0      0 :::49445                :::*  
udp6       0      0 :::5353                 :::*  
raw6       0      0 :::58                   :::*                   7  
Active UNIX domain sockets (servers and established)  
Proto RefCnt Flags               Type               State              I-Node    Path  
unix   3      [ ]               STREAM             CONNECTED          22671  
unix   3      [ ]               STREAM             CONNECTED          22545  
unix   3      [ ]               STREAM             CONNECTED          19151     /run/user/1000/bus  
unix   3      [ ]               STREAM             CONNECTED          20288     /run/user/1000/pipewire-0  
unix   3      [ ]               STREAM             CONNECTED          628       /run/systemd/journal/stdout  
unix   3      [ ]               STREAM             CONNECTED          19447
```

Figura 14. Verificando conexões e estado das portas com netstat.

1.2.3. Instalação/Configuração e acesso remoto via SSH

Agora que a conexão com a rede está funcionando, o próximo passo é instalar e configurar o SSH para permitir a comunicação entre as duas máquinas. No sistema host (Windows), ativamos o OpenSSH via Recursos Opcionais do Windows, enquanto na máquina virtual (guest), utilizamos o comando:

sudo apt-get install openssh-server

Com o SSH instalado, iremos realizar o acesso remoto à máquina virtual. No entanto, como estamos utilizando o modo de rede NAT no VirtualBox, o IP da máquina virtual não é acessível diretamente pelo host. Para resolver isso, configuramos o

redirecionamento de portas (Figura 9), permitindo o acesso via o seguinte comando na máquina hospedeira:

```
ssh vboxuser@localhost -p 22 ou ssh vboxuser@127.0.0.1 -p 22
```

Esse comando utiliza o redirecionamento de portas para conectar o host ao guest de maneira eficiente.

Ao executar o SSH no ambiente Windows, você encontrará uma mensagem relacionada à autenticação e segurança ao tentar se conectar a um servidor. Nesse caso, o aviso indica que a autenticidade do host localhost (127.0.0.1) não pôde ser confirmada. A ED25519 Key Fingerprint, apresentada como uma sequência única (SHA256), identifica a chave pública do servidor usando o algoritmo ED25519. Essa impressão digital garante que a chave do servidor não foi alterada e é consistente em conexões futuras.

A mensagem pergunta se você deseja continuar, pois o cliente SSH detectou uma nova chave para o host, que ainda não está na lista de "hosts conhecidos". Como é a primeira conexão com esse host, o cliente SSH informa que a chave será adicionada permanentemente à lista. Após aceitar (digitando "yes"), será solicitada a senha do usuário vboxuser (guest) para autenticar o acesso ao servidor, conforme mostrado nas Figura 15 e 16 abaixo.

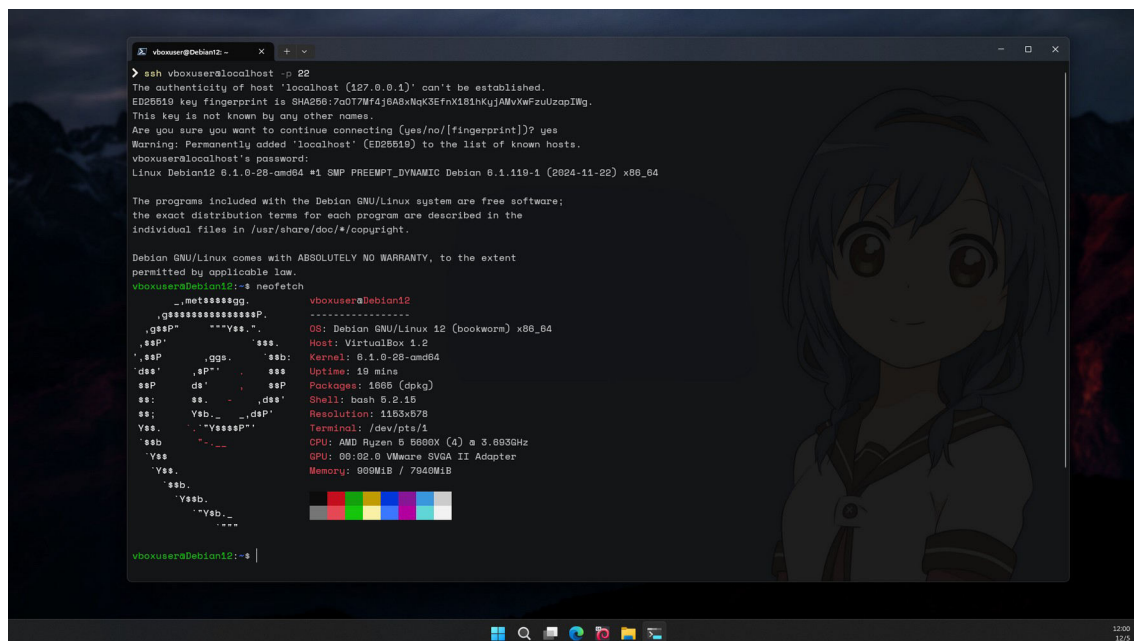


Figura 15. Conexão bem sucedida.

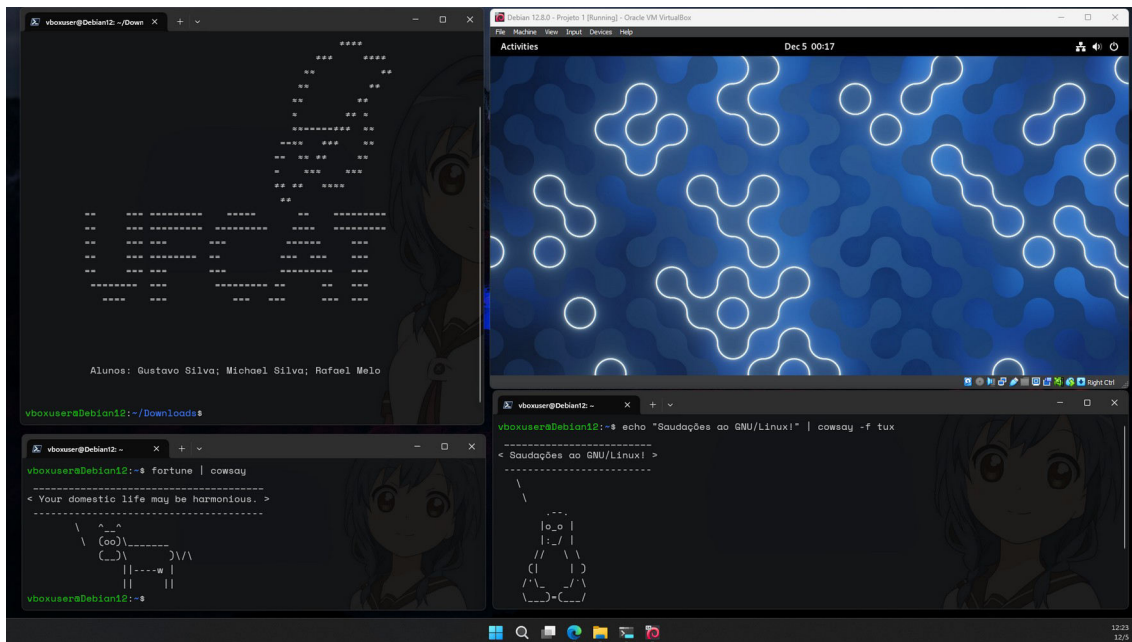


Figura 16. Usando 3 conexões ao mesmo tempo.

1.2.4. Transferência de Arquivos

Agora que temos tudo configurado, iremos transferir um arquivo do computador hospedeiro para a nossa Máquina Virtual (guest).

Para isso, pensando numa forma bem simples e estratégica, pensamos em criar um arquivo .txt no host utilizando o CMD como administrador e salvaremos ele em uma pasta teste localizada na raiz do disco C conforme o seguinte comando abaixo:

```
mkdir C:\Teste && cd C:\Teste
echo "Ola Mundo!" > teste.txt
cat .\teste.txt
```

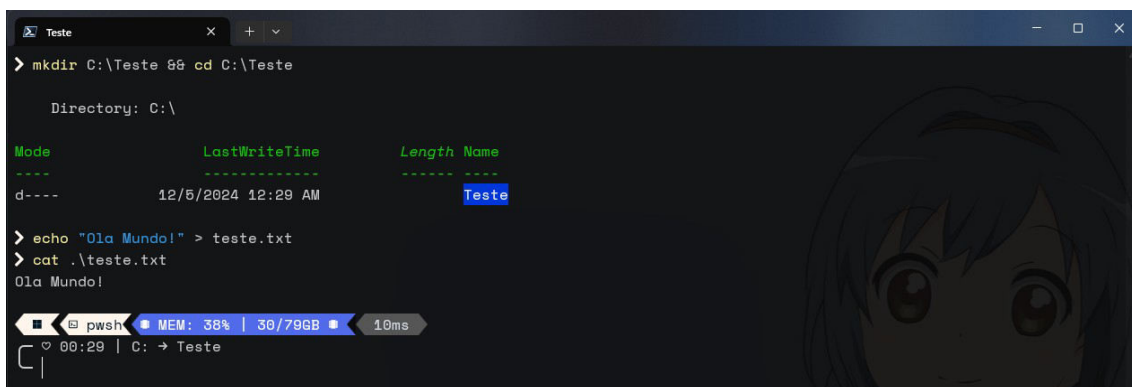


Figura 17. Preparando arquivo de transferência.

Após criarmos o arquivo, iremos transferi-lo para máquina virtual utilizando o

servidor de SSH recém instalado e configurado.

Para isto, usaremos o protocolo de comunicação SCP (Secure Copy Protocol) que utiliza o SSH (Secure Shell) como mecanismo subjacente para autenticação e criptografia. SCP permite a transferência segura de arquivos entre hosts em uma rede.

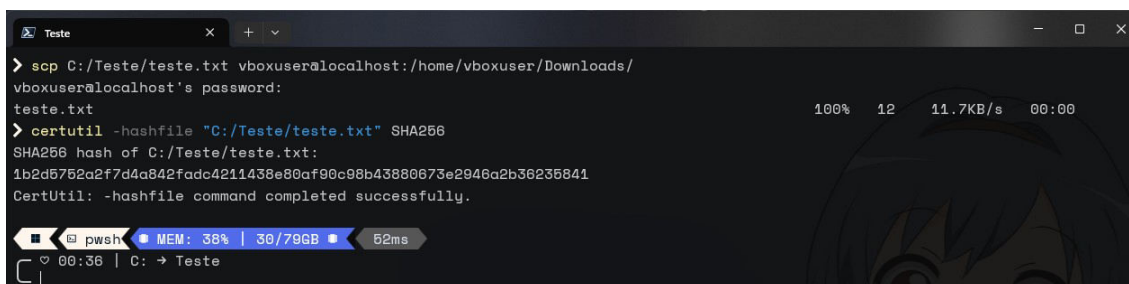
Quando usamos o comando `scp` para copiar arquivos, ele estabelece uma conexão SSH para autenticação e, em seguida, utiliza essa conexão para transferir os dados de forma segura. Assim, a criptografia e autenticação fornecidas pelo SSH também se aplicam às transferências de arquivos SCP.

Conquanto, em uma nova janela do terminal utilizaremos o seguinte comando:

```
scp C:/Teste/teste.txt vboxuser@localhost:/home/vboxuser/Downloads/
```

Será solicitada a senha e, após a inserção, o arquivo será copiado, conforme ilustrado na Figura 18. Entretanto, para conseguirmos provar que o arquivo é o mesmo, usaremos um comando no windows para gerar a hash do mesmo e depois avaliarmos no linux:

```
certutil -hashfile "C:/Teste/teste.txt" SHA256
```

A screenshot of a terminal window titled 'Teste'. The terminal shows the execution of an SCP command to copy a file from the local C:/Teste directory to the Downloads directory of a remote host (localhost) using the user 'vboxuser'. The password is entered as 'teste.txt'. Following the SCP command, the 'certutil' command is used to generate a SHA256 hash for the file 'C:/Teste/teste.txt'. The output shows the SHA256 hash value: '1b2d5752a2f7d4a842fadc4211438e80af90c98b43880673e2946a2b38235841'. The terminal also displays a progress bar for the SCP transfer, showing 100% completion, 12 files, and a transfer rate of 11.7KB/s. The bottom status bar shows 'pwsh', memory usage at 38%, and a 52ms execution time.

```
> scp C:/Teste/teste.txt vboxuser@localhost:/home/vboxuser/Downloads/
vboxuser@localhost's password:
teste.txt
> certutil -hashfile "C:/Teste/teste.txt" SHA256
SHA256 hash of C:/Teste/teste.txt:
1b2d5752a2f7d4a842fadc4211438e80af90c98b43880673e2946a2b38235841
CertUtil: -hashfile command completed successfully.
```

Figura 18. Protocolo SCP e hashfile.

Agora que o arquivo foi enviado para máquina guest, iremos navegar no diretório e verificar o hash do mesmo com o seguinte comando:

```
sha256sum Downloads/teste.txt
```

E iremos copiar qualquer um dos dois hash para procurarmos com `ctrl+shift+F` no terminal do host e comparar se as duas strings são iguais (Figura 19).

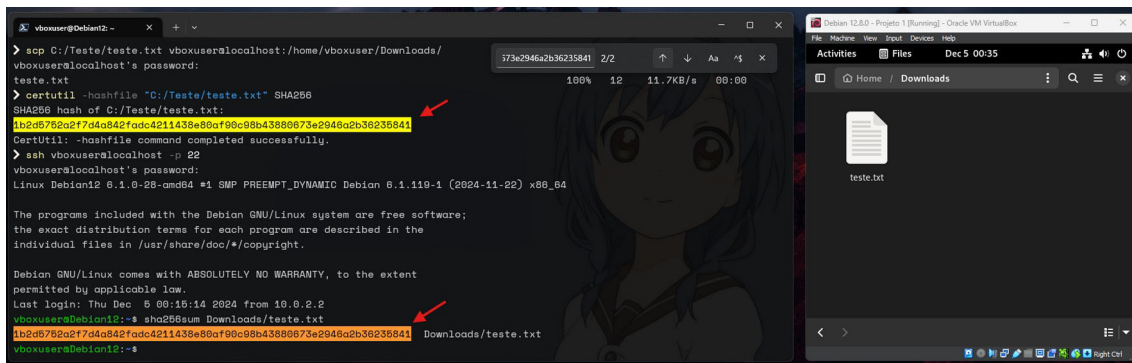


Figura 19. Protocolo SCP – Visão Geral.

1.2.5. Criação de Conta de Usuário

Agora iremos criar uma nova conta de usuário sem utilizar o root. E para isto usamos o comando **sudo adduser so2**.

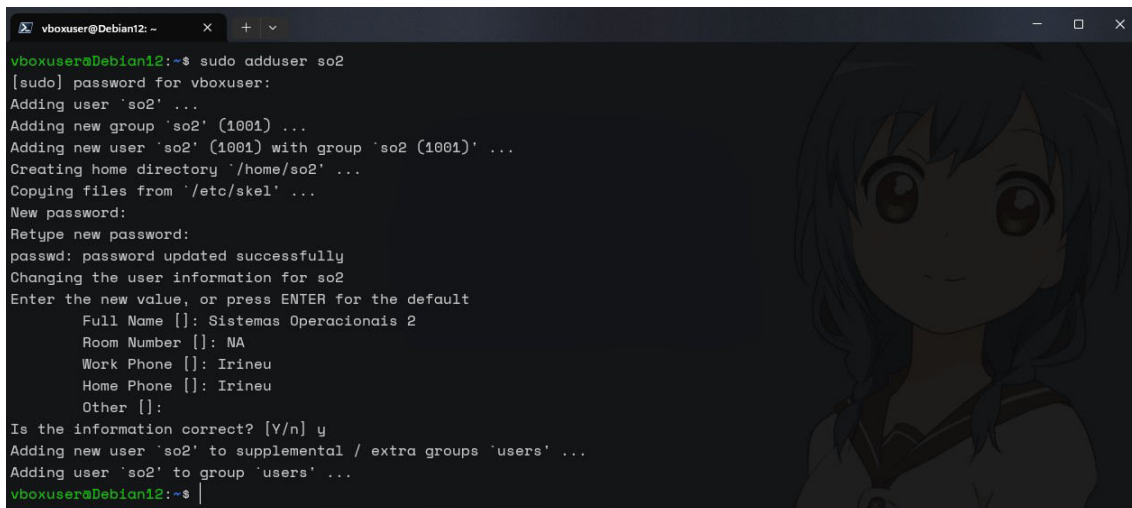


Figura 20. Adicionando novo usuário no guest.

Agora que está tudo Ok, vamos testar também a conexão SSH com novo usuário na máquina hospedeira repetindo o mesmo comando da Figura 21 porém com o novo usuário “so2”.

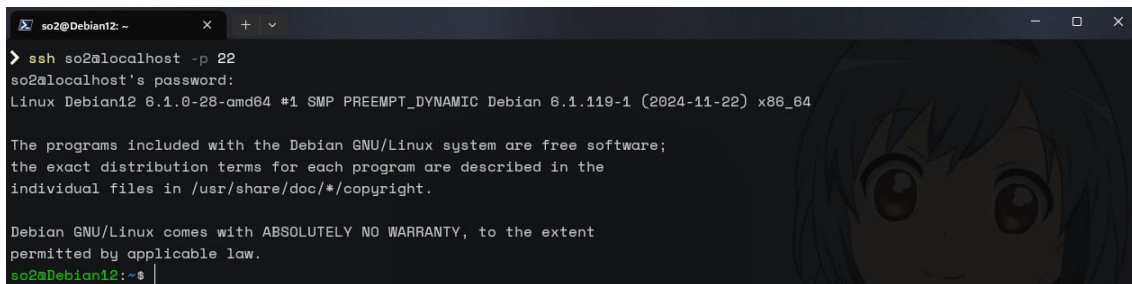
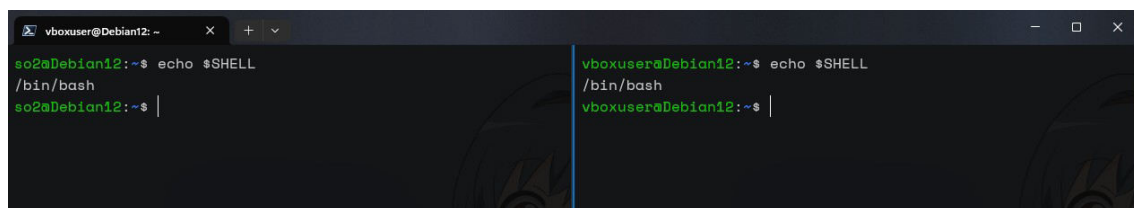


Figura 21. SSH com novo usuário conectado com sucesso.

1.2.6. Configuração do Bash ou zsh

Agora que criamos o usuário "so2", vamos configurar o shell padrão para esse novo usuário no sistema Linux. Essa operação é opcional, mas pode ser útil se o usuário preferir usar o Bash ou outro shell em vez do shell padrão definido durante a criação do usuário. No nosso caso, ambos os usuários parecem ter o /bin/bash configurado como shell padrão (Figura 22).



```
vboxuser@Debian12: ~  
so2@Debian12:~$ echo $SHELL  
/bin/bash  
so2@Debian12:~$  
vboxuser@Debian12:~$ echo $SHELL  
/bin/bash  
vboxuser@Debian12:~$
```

Figura 22. Shell atual dos usuários guest.

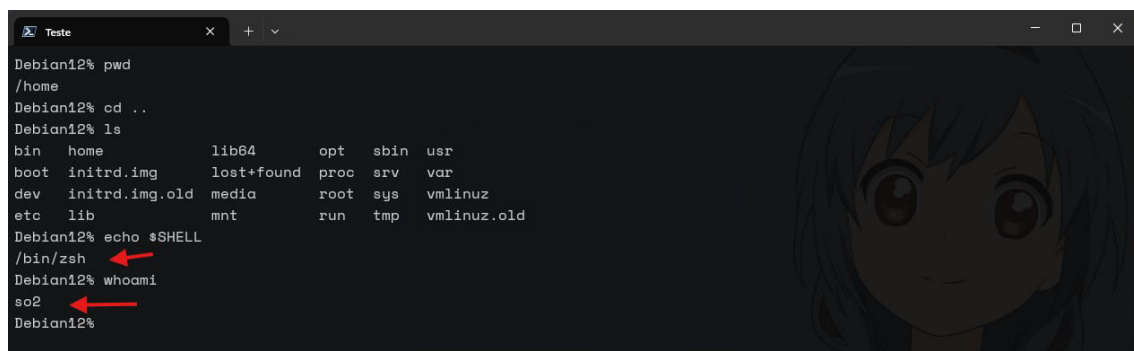
Para testar a configuração do shell, vamos alterar o shell do usuário "so2" para o zsh, com o objetivo de implementar o oh-my-zsh e o tema powerlevel10k. Para realizar essa alteração, precisaremos usar o usuário vboxuser, pois ele possui privilégios de superusuário, abaixo segue o procedimento realizado:

```
su vboxuser  
sudo apt install zsh  
sudo chsh -s /bin/zsh so2
```

Após isso, precisamos verificar se a alteração do shell do usuário so2 foi bem-sucedida através desse comando:

```
sudo grep so2 /etc/passwd
```

Ou simplesmente reabrindo o terminal do usuário so2 conforme a Figura 23 e executando o comando echo \$SHELL.



```
Debian12% pwd  
/home  
Debian12% cd ..  
Debian12% ls  
bin  home          lib64      opt       sbin      usr  
boot initrd.img     lost+found proc      srv       var  
dev  initrd.img.old media      root      sys      vmlinuz  
etc  lib           mnt       run       tmp      vmlinuz.old  
Debian12% echo $SHELL  
/bin/zsh  
Debian12% whoami  
so2  
Debian12%
```

Figura 23. Alteração do shell do usuário so2 bem-sucedida.

Para tornar mais fácil compreender as próximas ações, iremos organizar em passos.

Passos:

1. Antes de sair do usuário vboxuser, é interessante baixarmos as dependências necessárias para o oh-my-zsh, que no caso é curl e git:

```
sudo apt install curl git
```

2. Agora vamos voltar ao usuário so2 (usar o comando: su so2) e rodar esses comandos para baixarmos e instalarmos o tema powerlevel10k:

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git ~/powerlevel10k  
echo 'source ~/powerlevel10k/powerlevel10k.zsh-theme' >> ~/.zshrc
```

3. Agora vamos executar mais esse comando para baixarmos e instalarmos o oh-my-zsh:

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git $  
{ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k
```

4. Agora acessaremos o arquivo ~/.zshrc e, se ele não existir, vamos criar um novo:

```
nano ~/.zshrc
```

5. Dentro dele vamos procurar pela linha que contém ZSH_THEME e mudar o valor para “powerlevel10k/powerlevel10k” e se não existir a linha basta adicionar ela:

```
ZSH_THEME="powerlevel10k/powerlevel10k"
```

6. Após isso, vamos executar o comando abaixo para recarregar as configurações do zsh:

```
source ~/.zshrc
```

7. A partir desse momento, **iremos entrar na parte de configuração do powerlevel10k**, mas antes disso, tem que baixar alguma fonte que suporta símbolos da página nerd fonts (<https://www.nerdfonts.com/font-downloads>) e configura-la no terminal pois senão iremos ter problemas para passarmos no teste. Para o nosso teste, baixamos e instalamos a fonte “SpaceMono Nerd Font Mono” e a utilizamos no próprio terminal do windows conforme a Figura 24.

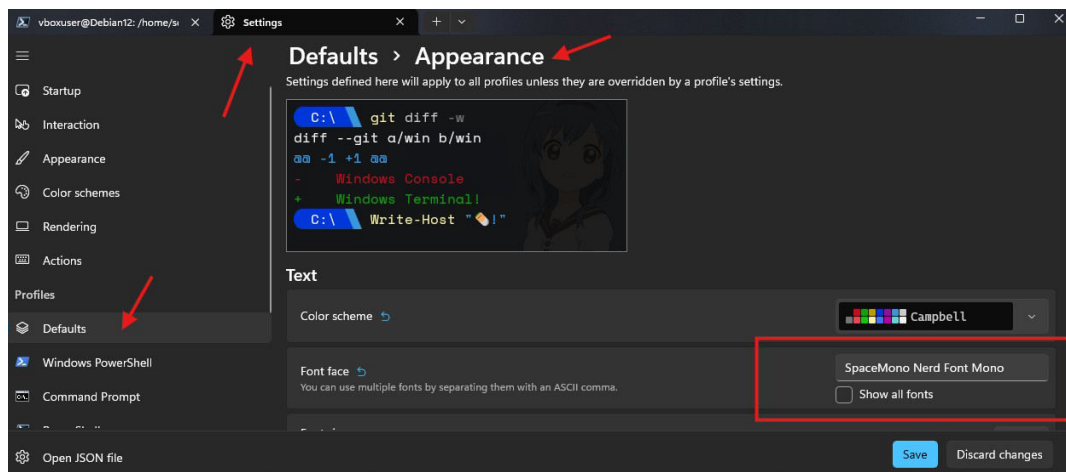


Figura 24. Atualização da fonte do terminal host.

8. Na Figura 25, podemos observar o menu inicial, o primeiro teste pergunta se estamos observando um diamante, basta responder conforme vai progredindo.

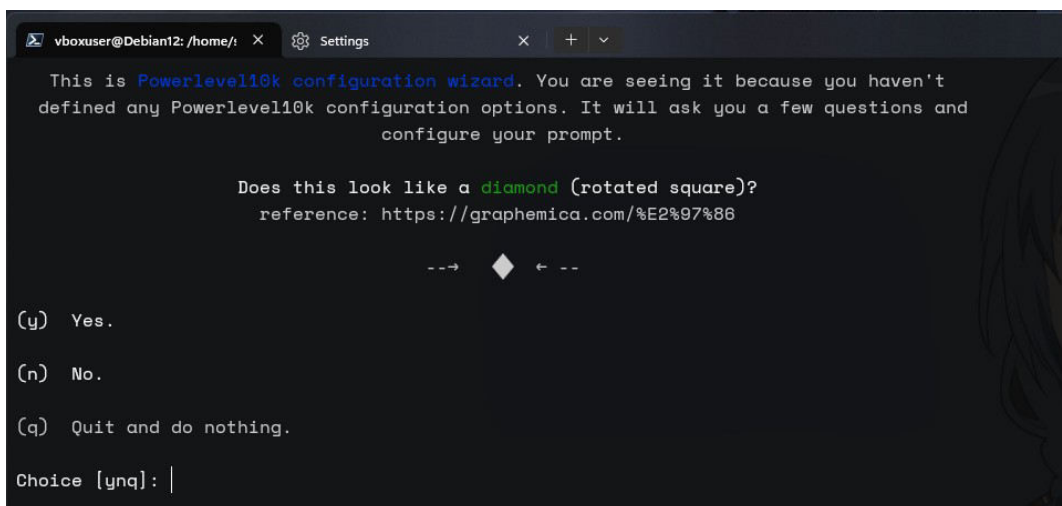


Figura 25. Menu powerlevel10k - Setup.

E ao chegar no fim, teremos na figura abaixo, o resultado final.

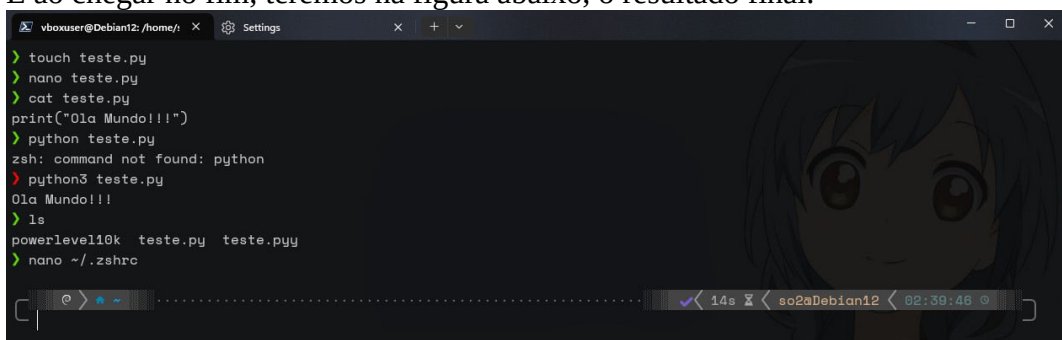


Figura 26. Setup finalizado.

1.3. Instalação de Softwares

Os softwares no Debian podem ser instalados de diversas maneiras, seja através da interface gráfica ou da linha de comando no terminal. A ferramenta principal para instalação de softwares é o apt, um poderoso sistema de gerenciamento de pacotes.

Na máquina Debian que utilizamos, a preferência foi pela instalação via linha de comando, utilizando o terminal. Isso proporcionou maior controle sobre o processo de instalação e permitiu uma abordagem mais personalizada para atender às necessidades específicas.

Em nenhum momento foi necessário adicionar repositórios de software para viabilizar a instalação de determinados softwares, pois o mesmo já atualizou automaticamente para gente durante a instalação. Isso possibilitou o acesso a uma variedade mais ampla de aplicativos e versões, garantindo a flexibilidade na escolha e personalização do ambiente de trabalho.

Essa abordagem, embora demande algum conhecimento adicional, revelou-se eficaz para garantir a disponibilidade e atualização dos softwares desejados, contribuindo para a estabilidade e desempenho do sistema operacional Debian.

2. Tarefa 2

2.1. Sincronização de processos

Essa tarefa tem como o objetivo recapitular conceitos fundamentais de sistemas operacionais, vistos em SO1. Nas seções a seguir serão apresentados os códigos desenvolvidos pelo nosso grupo.

2.1.1. Solução de Peterson

Para a realização desta atividade, foram utilizados como base os códigos apresentados pelo professor nos slides disponibilizados no Google Classroom.

A implementação da proteção da seção crítica segue as diretrizes fornecidas no enunciado e está de acordo com a abordagem proposta.

A estratégia de Peterson assegura a exclusão mútua na seção crítica. Isso é alcançado por meio de um protocolo que controla o acesso à seção crítica, utilizando variáveis de controle (flag e turn) para garantir que apenas um processo possa executá-la por vez. O código 1 implementa um loop de espera (busy-wait) que impede que um processo entre na seção crítica enquanto outro ainda está nela. Quando o processo libera a seção crítica, o loop é interrompido, permitindo que outro processo possa acessá-la.

Para lidar com possíveis erros, foi incorporada uma verificação condicional utilizando um if. Caso a função `pthread_create()` retorne um valor diferente de 0, isso indica um erro na criação da thread. Nesse caso, a execução do programa é encerrada, emitindo uma mensagem de erro.

A adaptação do código inclui a criação de duas threads, `pi` e `pj`, que utilizam

exclusivamente a função `void *cria(void *arg)`. Essa implementação foi baseada na documentação da função `pthread_create()` apresentada por Kerrisk (2023) [10].

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdbool.h> // Para usar true e false
#include <stdint.h> // Para usar intptr_t no cast de ponteiros

#define NUM_THREADS 2

bool flag[NUM_THREADS];
int turn, counter = 0;

void enterSC(int i) {
    int j = 1 - i; // J é o oposto de i
    flag[i] = true; // Thread solicita entrada na seção crítica
    turn = j;
    while (flag[j] && turn == j); // Laço para segurar a seção crítica
}

void leaveSC(int i) {
    flag[i] = false; // Thread sai da seção crítica
}

void *cria(void *arg) {
    int aux = (intptr_t)arg; // Converte para inteiro
    enterSC(aux); // Entra na seção crítica
    printf("P%d: A região crítica é minha!!!\n", aux);
    counter++;
    printf("P%d: counter = %d\n", aux, counter);
    leaveSC(aux); // Sai da seção crítica
    return NULL;
}

int main(void) {
    pthread_t threads[NUM_THREADS];

    // Criação das threads
    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, cria, (void *) (intptr_t) i) != 0) {
            fprintf(stderr, "Erro ao criar a thread %d\n", i);
            exit(1);
        }
    }

    // Aguarda as threads
    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_join(threads[i], NULL) != 0) {
            fprintf(stderr, "Erro ao aguardar a thread %d\n", i);
        }
    }
    return 0;
}
```

Código 1. Conteúdo do arquivo `peterson.c`.

Na figura (Figura 27) a seguir temos a output da execução do nosso código após

compilado com gcc (GNU Compiler Collection).

```
vboxuser@Debian12:~/Downloads$ gcc peterson.c -o peterson -lpthread
vboxuser@Debian12:~/Downloads$ ./peterson
P1: A região crítica é minha!!!
P1: counter = 1
P0: A região crítica é minha!!!
P0: counter = 2
—
```

Figura 27. Output do código.

2.1.2. Implementação de Locks "Test and Set" e Solução de Peterson em Hardware

Para realizar esta atividade foram usados como base o material disponibilizado pelo professor e também uma breve descrição das funções `atomic_flag_test_and_set` e `atomic_flag_clear`, descritos em cppreference.com [4, 5].

A implementação da proteção da seção crítica com locks do tipo "test and set" segue rigorosamente as diretrizes fornecidas no enunciado, utilizando o suporte de hardware para sincronização de threads. A abordagem emprega a biblioteca `pthread` para criação e gerenciamento de threads, em conjunto com as funções `atomic_flag_test_and_set` e `atomic_flag_clear` da biblioteca `stdatomic.h`, que fornecem um mecanismo eficiente e seguro para exclusão mútua.

O uso do `atomic_flag` assegura a exclusão mútua na seção crítica. Isso é realizado por meio de uma variável de intertravamento (lock) inicializada com `ATOMIC_FLAG_INIT`. O controle de acesso à seção crítica é gerenciado por meio da função `atomic_flag_test_and_set`, que realiza uma operação atômica para verificar e bloquear o acesso. Enquanto o lock está ocupado, as outras threads permanecem em um loop de espera (busy-wait), impedindo a entrada na seção crítica. Após a conclusão da seção crítica, a função `atomic_flag_clear` é usada para liberar o lock, permitindo que outras threads possam acessá-la.

Para lidar com possíveis erros, foi incorporada uma verificação condicional em cada chamada à função `pthread_create()`. Caso ela retorne um valor diferente de 0, isso indica uma falha na criação da thread. Nesse caso, o programa emite uma mensagem de erro e encerra a execução.

A adaptação do código inclui a criação de duas threads, que acessam a seção crítica de maneira sincronizada. Cada thread utiliza a função `void *cria(void *arg)` para executar suas operações. Essa implementação é baseada na documentação da função `pthread_create()` e nas descrições técnicas do uso de `atomic_flag`, conforme recomendado no enunciado e em materiais de referência.

O código 2, demonstra de forma prática o controle de acesso sincronizado, garantindo a integridade dos dados compartilhados e evitando condições de corrida (race conditions) que seria basicamente quando dois ou mais processos ou threads tentam acessar ou modificar dados compartilhados de maneira simultânea, sem a devida sincronização, o que pode levar a resultados imprevisíveis ou incorretos.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdatomic.h> // Biblioteca para operações atômicas
#include <stdint.h> // Para usar intptr_t

int counter = 0; // Variável compartilhada inicializada
atomic_flag lock = ATOMIC_FLAG_INIT; // Variável atômica para o lock

// Função para entrar na seção crítica
void enterSC(int i) {
    while (atomic_flag_test_and_set(&lock)) {
        // Laço para travar a seção crítica
    }
    return;
}

// Função para sair da seção crítica
void leaveSC(int i) {
    atomic_flag_clear(&lock); // Libera a seção crítica
    return;
}

// Função executada pelas threads
void* cria(void* arg) {
    int aux = (intptr_t)arg; // Converte o argumento para inteiro de maneira segura

    // Entra na seção crítica
    enterSC(aux);
    printf("\nP%d: A região crítica é minha!!!\n", aux);
    counter++;
    printf("\nP%d: counter = %d\n", aux, counter);
    leaveSC(aux); // Sai da seção crítica

    return NULL;
}

// Função para verificar erros na criação das threads
void verifica_erro(int resultado) {
    if (resultado != 0) {
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }
}

int main(void) {
    pthread_t thi, thj; // Variáveis para as threads

    // Criação das threads com verificação de erros
    verifica_erro(pthread_create(&thi, NULL, cria, (void*)0));
    verifica_erro(pthread_create(&thj, NULL, cria, (void*)1));

    // Espera as threads terminarem
    pthread_join(thi, NULL);
    pthread_join(thj, NULL);

    return 0;
}

```

Código 2. Conteúdo do arquivo atomic.c.

```
vboxuser@Debian12:~/Downloads$ gcc atomic.c -o atomic -lpthread
vboxuser@Debian12:~/Downloads$ ./atomic

P0: A região crítica é minha!!!

P0: counter = 1

P1: A região crítica é minha!!!

P1: counter = 2
```

Figura 28. Output do código.

2.1.3. Solução de hardware usando locks do tipo mutex

A implementação da proteção da seção crítica com locks do tipo mutex segue as diretrizes fornecidas no enunciado e está de acordo com a abordagem proposta. O mutex (mutual exclusion) é utilizado para garantir que apenas uma thread tenha acesso à seção crítica de cada vez, prevenindo problemas como race conditions. A biblioteca pthread é usada para a criação e sincronização das threads, e as funções pthread_mutex_lock e pthread_mutex_unlock são responsáveis por travar e liberar a seção crítica.

O código 3 utiliza uma variável do tipo pthread_mutex_t chamada lock para controlar o acesso à seção crítica. A função pthread_mutex_lock é chamada para travar o mutex, bloqueando a entrada de outras threads na seção crítica, enquanto a função pthread_mutex_unlock é utilizada para liberar o mutex, permitindo que outras threads acessem a seção crítica após a conclusão da execução da thread atual.

O controle de acesso à seção crítica é realizado nas funções enterSC() e leaveSC(). A função enterSC() bloqueia a entrada na seção crítica, enquanto leaveSC() a libera após a execução das operações críticas. Na função cria(), cada thread entra e sai da seção crítica ao chamar essas funções, garantindo que o contador seja incrementado de forma sincronizada e sem conflitos.

Para garantir a robustez do código, foi incorporada uma verificação condicional em cada chamada de pthread_create(). Caso a função retorne um valor diferente de zero, isso indica um erro na criação da thread, e o programa exibe uma mensagem de erro e é encerrado.

A adaptação do código inclui a criação de duas threads, thi e thj, que acessam a seção crítica de maneira sincronizada. Cada thread executa a função cria(), onde realiza suas operações na seção crítica de forma controlada, utilizando o mutex para garantir que o contador seja modificado de maneira segura.

Esse código demonstra de forma prática o uso de *mutexes* para controlar o acesso à seção crítica, assegurando que o acesso seja sequencial e que a integridade dos dados compartilhados seja preservada, evitando condições de corrida.

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t lock; // Variável para travamento da SC
int counter = 0; // Contador

void enterSC() {
    pthread_mutex_lock(&lock); // Trava a seção crítica pela variável lock
    return;
}

void leaveSC() {
    pthread_mutex_unlock(&lock); // Libera a SC pela variável lock
    return;
}

void* cria(void* arg) {
    int aux = *(int*)arg, i = 0; // Pegar o índice da thread corretamente
    enterSC(); // Chamada da função para entrar na seção crítica
    printf("\nP%d: A região crítica é minha!!!\n", aux);
    counter++;
    printf("\n\nP%d: counter = %d\n\n", aux, counter);
    leaveSC(); // Função para sair da seção crítica
    return NULL;
}

int main(void) {
    pthread_t thi, thj; // Duas variáveis para a criação das threads

    // Verifica se a thread foi criada corretamente
    int arg1 = 0, arg2 = 1; // Definindo argumentos para passar para as threads
    if (pthread_create(&thi, NULL, cria, &arg1) != 0) {
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }

    if (pthread_create(&thj, NULL, cria, &arg2) != 0) {
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }

    // Inicia as threads
    pthread_join(thi, NULL);
    pthread_join(thj, NULL);

    return 0;
}

```

Código 3. Conteúdo do arquivo mutex.c.

```
vboxuser@Debian12:~/Downloads$ gcc mutex.c -o mutex -lpthread
vboxuser@Debian12:~/Downloads$ ./mutex

P0: A região crítica é minha!!!

P0: counter = 1

P1: A região crítica é minha!!!

P1: counter = 2
```

Figura 29. Output do código.

2.1.4. Solução de hardware usando semáforo

Para a implementação desta tarefa, utilizei os materiais fornecidos pelo instrutor, incluindo os slides da disciplina de Sistemas Operacionais I (SO1), além de consultar fontes externas para uma descrição detalhada das funções de semáforo [6, 7, 8, 9]. O objetivo desta solução é controlar o acesso à seção crítica utilizando semáforos, uma ferramenta comumente utilizada em sistemas operacionais para garantir a exclusão mútua e evitar condições de corrida (race conditions).

O código 4 foi estruturado para utilizar semáforos como mecanismo de sincronização entre as threads. Para isso, a biblioteca pthread foi usada para a criação e gerenciamento das threads, enquanto a funcionalidade de semáforo foi implementada por meio da biblioteca semaphore.h. O semáforo utilizado no código é uma variável do tipo sem_t, responsável por controlar o acesso à seção crítica, garantindo que apenas uma thread por vez tenha permissão para modificar o recurso compartilhado.

A função enterSC() é responsável por bloquear o semáforo, impedindo que outras threads acessem a seção crítica até que a thread atual libere o semáforo. A chamada sem_wait(&semaphore) faz com que a thread que a executa aguarde até que o semáforo esteja disponível. Já a função leaveSC() libera o semáforo, permitindo que outras threads acessem a seção crítica ao chamar sem_post(&semaphore).

As threads são criadas com a função pthread_create(), e cada thread executa a função cria(), que inclui a entrada e saída da seção crítica. A função cria() imprime o identificador da thread e o valor do contador compartilhado após incrementá-lo, garantindo que a modificação do contador seja feita de forma segura e sem interferência de outras threads, graças ao controle proporcionado pelo semáforo.

O semáforo é inicialmente configurado com o valor 1 por meio da função sem_init(&semaphore, 0, 1), o que permite que apenas uma thread entre na seção crítica por vez. Após a execução das threads, o programa utiliza pthread_join() para aguardar a finalização de ambas as threads e, ao final, chama sem_destroy(&semaphore) para destruir o semáforo, liberando os recursos utilizados.

Essa solução utilizando semáforos é uma abordagem eficiente e simples para

resolver o problema de controle de acesso a recursos compartilhados em ambientes multithread. O uso do semáforo garante que o acesso à seção crítica seja exclusivo para uma thread por vez, evitando condições de corrida e garantindo a integridade dos dados compartilhados.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h> // Biblioteca para o uso de semáforos

sem_t semaphore; // Variável de semáforo
int counter = 0;

void enterSC() { // Função para controlar entrada na seção crítica
    sem_wait(&semaphore); // Para fazer o lock do semáforo
    return;
}

void leaveSC() { // Função para sair da seção crítica
    sem_post(&semaphore); // Função que libera o semáforo
    return;
}

void* cria(void* arg) { // Função para a thread
    int aux = (int)(intptr_t)arg; // Corrigindo o cast do ponteiro para inteiro
    enterSC(); // Chamada da função para entrar na seção crítica
    printf("\nP%d: A região crítica é minha!!!\n", aux);
    counter++;
    printf("\n\nP%d: counter = %d\n\n", aux, counter);
    leaveSC(); // Função para sair da seção crítica
    return NULL;
}

int main(void) {
    pthread_t thi, thj; // Duas variáveis para a criação das threads
    // Inicializa o semáforo com valor 1 (permitindo que uma thread entre por vez)
    sem_init(&semaphore, 0, 1);

    // Verifica se a thread foi criada corretamente
    if (pthread_create(&thi, NULL, cria, (void*)(intptr_t)0) != 0) {
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }
    if (pthread_create(&thj, NULL, cria, (void*)(intptr_t)1) != 0) {
        printf("Algum erro ocorreu na criação das threads!\n");
        exit(1);
    }
    // Inicia as threads
    pthread_join(thi, NULL);
    pthread_join(thj, NULL);

    sem_destroy(&semaphore); // Destrói o semáforo ao fim do programa
    return 0;
}
```

Código 4. Conteúdo do arquivo semaphore.c.

```

vboxuser@Debian12:~/Downloads$ gcc semaphore.c -o semaphore -lpthread
vboxuser@Debian12:~/Downloads$ ./semaphore

P0: A região crítica é minha!!!

P0: counter = 1

P1: A região crítica é minha!!!

P1: counter = 2

```

Figura 30. Output do código.

3. Tarefa 3

Para resolver esta tarefa, utilizamos a abordagem de semáforos discutida na seção 2.1.4 como referência. Desenvolvemos uma simulação na qual dois processos, P0 e P1, são executados simultaneamente. Durante a execução, os semáforos S e Q são utilizados para controlar os recursos compartilhados.

No início, o processo P0 solicita o recurso S, enquanto o P1 solicita o recurso Q. Ambos realizam um pequeno intervalo de espera (utilizando a função `sleep` da biblioteca `unistd`) para simular um processamento antes de tentarem acessar o próximo recurso. Após isso, P0 tenta adquirir Q, que já foi alocado por P1, e simultaneamente P1 tenta adquirir S, que está em uso por P0. Como resultado, ambos os processos entram em um estado de espera mútua, caracterizando um impasse (deadlock) conforme demonstrado na Figura 31.

Essa implementação permite ilustrar claramente a ocorrência de um deadlock em um cenário simples com dois processos e dois recursos. O código resultante está apresentado a seguir.

```

#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>
#include <semaphore.h>

pthread_t tid[2];
sem_t S, Q; // Semáforos S e Q, representando os recursos compartilhados

void* P0(void* arg) {
    printf("P0 tentando adquirir o recurso S...\n"); // Debug
    sem_wait(&S); // P0 adquire o semáforo S
    printf("P0 adquiriu o recurso S.\n"); // Debug
    sleep(1); // Simula processamento
    printf("P0 tentando adquirir o recurso Q...\n"); // Debug
    sem_wait(&Q); // P0 tenta adquirir Q, mas já foi adquirido por P1
    printf("P0 adquiriu o recurso Q.\n"); // Debug

    // Código crítico
    printf("P0 entrou na região crítica.\n");
}

```

```

sem_post(&Q); // Libera Q
printf("P0 liberou o recurso Q.\n"); // Debug
sem_post(&S); // Libera S
printf("P0 liberou o recurso S.\n"); // Debug
return NULL;
}

void* P1(void* arg) {
printf("P1 tentando adquirir o recurso Q...\n"); // Debug
sem_wait(&Q); // P1 adquire o semáforo Q
printf("P1 adquiriu o recurso Q.\n"); // Debug
sleep(1); // Simula processamento
printf("P1 tentando adquirir o recurso S...\n"); // Debug
sem_wait(&S); // P1 tenta adquirir S, mas já está ocupado por P0
printf("P1 adquiriu o recurso S.\n"); // Debug

// Código crítico
printf("P1 entrou na região crítica.\n");

sem_post(&S); // Libera S
printf("P1 liberou o recurso S.\n"); // Debug
sem_post(&Q); // Libera Q
printf("P1 liberou o recurso Q.\n"); // Debug
return NULL;
}

int main() {
sem_init(&S, 0, 1); // Inicializa o semáforo S com valor 1 (recurso disponível)
sem_init(&Q, 0, 1); // Inicializa o semáforo Q com valor 1 (recurso disponível)

pthread_create(&tid[0], NULL, P0, NULL); // Cria a thread P0
pthread_create(&tid[1], NULL, P1, NULL); // Cria a thread P1

pthread_join(tid[0], NULL); // Aguarda P0 terminar
pthread_join(tid[1], NULL); // Aguarda P1 terminar

sem_destroy(&S); // Destrói o semáforo S
sem_destroy(&Q); // Destrói o semáforo Q

return 0;
}

```

Código 5 Conteúdo do arquivo `deadlock_simulacao_semaforos.c`.

Um adendo, para rodar o código e ficar mais fácil para compilar, criamos um arquivo Makefile (Código 6) na pasta onde está localizado o código 5, para compilar e executar logo em seguinte, basta usar:

```
make && ls && ./deadlock_simulacao_semaforos
```

```

# Definições
CC = cc
LIBS = -lpthread
FLAGS = -O -w
EXECS = deadlock_simulacao_semaforos

# Regra principal

```

```

all: $(EXECS)

# Regra para compilar o código de deadlock
deadlock_simulacao_semaforos: deadlock_simulacao_semaforos.c
    $(CC) $(FLAGS) -o $@ $? $(LIBS)

# Regra para limpar os arquivos gerados
clean:
    rm -f $(EXECS)
}

```

Código 6 Conteúdo do arquivo Makefile.

```

make
zsh MEM: 12.43% | 1/7GB 12ms
08:51 | → Codigos
make 88 ls 88 ./deadlock_simulacao_semaforos
cc -O -w -o deadlock_simulacao_semaforos deadlock_simulacao_semaforos.c -lpthread
deadlock_simulacao_semaforos deadlock_simulacao_semaforos.c Makefile
P0 tentando adquirir o recurso S...
P0 adquiriu o recurso S.
P1 tentando adquirir o recurso Q...
P1 adquiriu o recurso Q.
P0 tentando adquirir o recurso Q...
P1 tentando adquirir o recurso S...

```

Figura 31. Output do código.

O código 5 simula a ocorrência de um deadlock utilizando semáforos para gerenciar dois recursos compartilhados (S e Q). No início da execução, os processos P0 e P1 tentam adquirir os semáforos para acessar os recursos. O processo P0 tenta adquirir o recurso S e, em seguida, o recurso Q. Simultaneamente, o processo P1 tenta adquirir o recurso Q e, depois, o recurso S.

Devido à ordem em que os recursos são adquiridos, ambos os processos entram em um impasse (deadlock). P0 está esperando que P1 libere o recurso Q, enquanto P1 está esperando que P0 libere o recurso S. Como ambos os processos estão esperando um pelo outro, nenhum pode continuar, resultando em um deadlock.

A lógica do código foi criada para demonstrar esse comportamento de deadlock de forma simples. Esse tipo de situação é uma preocupação comum em sistemas concorrentes e geralmente é evitada por meio de técnicas de sincronização, como garantir uma ordem de aquisição de recursos ou detectar e recuperar de deadlocks.

Output esperado:

- P0 tenta adquirir S, o que é bem-sucedido.
- P1 tenta adquirir Q, o que também é bem-sucedido.
- P0 tenta adquirir Q, mas está bloqueado, pois P1 já o possui.
- P1 tenta adquirir S, mas está bloqueado, pois P0 já o possui.

No final, ambos os processos estão esperando por um recurso que o outro possui, resultando em um deadlock.

4. Tarefa 4

...

5. Referências

- [1]. Debian, (2023) “Instalando o Debian via Internet”, <https://www.debian.org/distrib/netinst>, Junho.
- [2]. Vinicius Andrade Ribeiro, (2019) “Instalar o Debian 10.0 no VirtualBox 5.2.22”, <https://eteourinhos.com.br/wp-content/uploads/2019/08/02-Instalar-o-Debian-10.pdf>, Agosto.
- [3]. Gustavo Sobreira, Gabriel Fernandes, Fórum Alura (2022) “[Sugestão] [Resolvido] user is not in the sudoers file. this incident will be reported”, <https://cursos.alura.com.br/forum/topico-sugestao-resolvido-user-is-not-in-the-sudoers-file-this-incident-will-be-reported-259842>, Dezembro.
- [4]. ATOMIC_FLAG_CLEAR (2022) “atomic_flag_clear_explicit”. https://en.cppreference.com/w/c/atomic/atomic_flag_clear, Março.
- [5]. ATOMIC_FLAG_TEST_AND_SET (2022) “atomic_flag_test_and_set_explicit”. https://en.cppreference.com/w/c/atomic/atomic_flag_test_and_set, Março.
- [6]. [SEM_WAIT, Michael Kerrisk 2023], “sem_wait(3) — Linux manual page”. http://man7.org/linux/man-pages/man3/sem_wait.3.html, Março.
- [7]. [SEM_POST, Michael Kerrisk 2023], “sem_post(3) — Linux manual page”. http://man7.org/linux/man-pages/man3/sem_post.3.html, Março.
- [8]. [SEM_INIT, Michael Kerrisk 2023], “sem_init(3) — Linux manual page”. http://man7.org/linux/man-pages/man3/sem_init.3.html, Março.
- [9]. [SEM_DESTROY, Michael Kerrisk 2023], “sem_destroy(3) — Linux manual page”. http://man7.org/linux/man-pages/man3/sem_destroy.3.htm, Março.
- [10]. [PTHREAD_CREATE, Michael Kerrisk 2023], “pthread_create(3) — Linux manual page”. http://man7.org/linux/man-pages/man3/pthread_create.3.html, Março.